

## CHAPTER 1

### INTRODUCTION

P?S/8 is an operating system for the PDP-8 and related computers. It includes a variety of system-specific utilities and supports several programming languages. The system has been under sporadic development for many years including some recent upgrades.

This document details major events in the development of P?S/8, which supports configurations for any 12-bit computer for the "family of 8" subset of the Digital Equipment Corporate (DEC) PDP-8 models (and follow-on computers that are sufficiently compatible including DECmates and products of certain third-party vendors). Supported peripheral devices and other issues will be discussed as necessary.

Much of the details of this document are obtained from the recollections of Charles Lasner, the overall main architect of the entire P?S/8 project. That said, much of the specific features were implemented using the works of others. Where important to the narrative, certain parties will be mentioned either specifically or categorically. The proper credit (or in some cases blame) will be included for proper clarity. Wherever possible, relevant source files including edit dates will be relied on for accuracy.

#### 1.1 Overall Description

The PDP-8 computer represents a major milestone in the development of computers; it is arguably the first computer model that represents a serious departure from former ideas regarding a centralized large computer that is hardly approachable; users of these earlier systems had only indirect contact with the computer in any sense. Interaction with the computer was not only unavailable, attempts to allow this to be overcome were generally met with skepticism if not total denial of the proper role of a computer. Users were relegated to using peripheral equipment such as the use of manually operated computer keypunch equipment; the machine cycles were considered "too important" or "too valuable" to be used for anything other than calculations.

An appropriate starting point for the development of small computer systems with an emphasis on user interaction is the 1960 introduction of the 12-bit CDC 160 designed by Seymour Cray; while Cray is primarily associated with various large computer designs, he was known to have realized the future of computing was in direct interaction with smaller systems.

The CDC 160, while not a complete success for various technical reasons at least attempted to create an environment a computer programmer or user could physically sit down at; the CPU and memory cards were built into a desk. While a good start, the rest of the actual 160 system was still a series of bulky peripherals requiring personnel to tend to: Card readers required an operator to have a stack of cards read into computer memory started by commands typed at the desk. Various operations required mounting magnetic tapes or punching card decks on bulky card punches. All of these devices required an operator (presumably the user frequently interrupted; this is a hold-over from the larger computers that came before).

The CDC 160 was an inspiration to others in the computing industry, and the next logical step is the MIT Lincoln Laboratories small computer known as the LINC (Laboratory INstrument Computer).

The LINC is nearly what would define the beginning of personal computing in many ways: the computer included small block-replaceable storage tapes that could be placed into a pocket when not mounted on the drives; these storage tapes, known as LINCtapes, represented a very reliable device that was practical for a relatively inexpensive computer. Peripherals geared to laboratory work such as Analog-Digital convertors are included, as well as a built-in analog display on a specially modified Tektronix oscilloscope are standard equipment.

The first interactive software for use by the somewhat specialized user audience (consisting of researchers with at least one assembly language programmer) allowed intrepid computer programmers to write small programs for the benefit of the small group of associated researchers; the physical size of the computer, while somewhat large in the overall sense, was a lot smaller than the more typical large computers of the previous era.

While the LINC computer taken as a complete system of hardware and software was undoubtedly some form of conceptual "first" for the era (1962), it fails to be adequate to promote more general usage of computers in part because the software is geared to development of small programs in assembly language exclusively; the only other commands are concerned either with the primitive aspects of basic file management or merely running the binary version of the program.

Certain implementation techniques used are quite primitive making these software systems needlessly slow.

Note: By way of comparison, most successful computer systems require three stages of assembly language file support: a), the source program itself, b) binary files created by assembling the source program and c) a means to load these binary programs using a loading utility. The LINC system and the various follow on systems did not support a loading program; instead the assembly process is inefficient as the supported format is a direct image of memory; while this is easier to load once developed, the assembly process must inefficiently process binary output as a tentative image of memory with boundary requirements.

This was done in a short-sighted manner to accommodate certain of the limitations of LINCtapes. However, there are other ways to accomplish this on equivalent devices without the attendant overhead. This is part of the design of the PDP-8 systems that followed, which were much more popular and had standards of software design that were overlooked by the somewhat naive community of LINC users; they were mostly interested in the ability to run their programs and not the limitations of the environment used to create the running programs. Additionally, only the actual programmers could have any knowledge to even know what the issues are; as computers evolved, the user community became more savvy and could draw conclusions as to software efficacy.

An important criticism of the LINC architecture is that it is compromised by its inadequate memory size. The LINC was originally designed as a 1K memory system with limited capability; this was later upgraded to a 2K design where the second 1K of memory can only be accessed in limited ways. The design is not adequate for more demanding applications that would come later on the PDP-8. While clearly a first in terms of small-scale and user approachability, this system was essentially a dead-ended concept that none-the-less provided many good ideas.

In 1963, there was an abortive attempt at what would be later known as the PDP-8. This machine was the PDP-5, the first 12-bit computer from DEC. As the LINC before it, the main components were DEC "System" modules which are rather large considering how little logic is contained on any one of them; several other computers were also constructed from these clumsy modules until DEC abandoned them in favor of far smaller modules first identified (somewhat erroneously) as Flip-Chip modules.

Note: Flip-Chip modules are essentially equivalent in most ways to the far larger System modules, but are about  $\frac{1}{8}$  the overall size. Often similar components appear on the Flip-Chip circuit boards; however, there is far less empty space on the newer boards in the majority of cases. When hundreds of modules are used within a computer design, the amount of overall space saved is considerable; moreover, the backplanes of Flip-Chip modules were wire-wrapped and not soldered. This allowed numerically-controlled wire-wrap machines to create backplanes with more reliability without manual labor, and in far less overall production time. In certain versions of some of the modules, individual electronic components were replaced with small-scale integration of a few analog components each. These modestly integrated components are the actual "Flip-Chip" parts, but DEC marketing chose to call the entire module by that name. In fact, many of the modules never actually used the "Flip-Chip" parts.

Had the PDP-5 stuck to the original proposed design that became the PDP-8, it might have been successful. Unfortunately the design was bastardized in several ways leading to an unsuccessful product that was purchased by far too few customers to be successful.

The PDP-8 design is inherently 4096 12-bit words (4K) upgradable in increments of 4K to a maximum of 32K. (Upgrades were expensive, especially in the earliest models.) In a desperate attempt to lower the perceived price, marketing material quoted unrealistic minimum configurations that cost far too much additional money to create the desired ones. Instead of a 4K machine, the PDP-5 was marketed as a 1K machine, upgradable by very expensive means to 4K. The design of the memory extension beyond 4K was essentially abandoned along with some other prematurely announced features that were never implemented.

Because of the unwieldy nature of an unbalanced upgrade, this was especially costly; however, the actual cost increase was partially obfuscated by another ill-conceived notion.

In virtually all other computer designs, the basic components of a central processor include at least one arithmetic register (called the accumulator, which also has a programmable carry bit known as the Link bit), a memory address register and a memory buffer register; the latter two registers are necessary to access the memory. (The issues raised above regarding the memory implementation are a more nuanced design decision.)

While all of the above requirements are met in the PDP-5, there is one extremely poor element of the design: while there should be a program counter to keep track of where the program is operating, they chose to instead designate memory location 0000 in lieu of the operating hardware of such a register. This implementation requires a series of extra operating states as the contents of location 0000 is ferried in and out of memory location 0000 and into the memory address register.

The design decision is an attempt to save the cost of 12 additional register modules (which are admittedly somewhat more expensive than gating modules). However, the overall method causes the typical overall cycle time of every instruction to be seriously slow compared to a more conventional design.

The problem is that each of these cycles has to be carried out at a relatively high speed, thus the PDP-5 uses a very expensive faster core memory than would otherwise be used. The net combination is arguably more expensive or of inadequate economic reasoning; regardless, the computer is quite slow. (Note: The PDP-8 design uses a similar-speed core memory module but runs five times as fast!)

Thus, with all of these short-sighted blunders, not only was the PDP-5 an economic disaster for DEC, it nearly prevented the later creation of the PDP-8 entirely. (It should be noted that a lot of these issues were hot topics of discussion among various senior engineers and top management; the negative interactions are in part the subject of certain books that attempt to explain the demise of DEC due to poor management techniques.)

The PDP-8 was introduced in 1965 as an economy priced small computer, complete with 4K of memory and an attached Teletype Model 33 ASR (which allows binary paper-tapes to be slowly read in or slowly punched to blank paper-tape) Very little software was provided at that time because it would be some time before storage peripherals appeared for the machine.

At that point, the only software was very limited, awkward and clumsy to use, but considering the limited environment, about as much as could be expected.

Sometime later, various of the personnel of the LINC project (in some cases moonlighting there while DEC employees and using DEC's design facilities) were hired to create several internal projects for DEC, including DEC constructing and selling complete LINC systems to a few customers; the overall quantity of DEC-made LINC systems is estimated at approximately 50 units. (This also allowed DEC to work with the vendor of some of the original LINC tape drives, which were in part copied in DEC production drives released later, such as the TU55 DECTape drive, also used for LINCtapes on the PDP-12.)

Despite being economically unsound, a DECTape peripheral was constructed for the PDP-5 bus (with problematic support for extended memory, which was not a concern for the few potential customers). It is not clear if any were actually sold or just used in-house on prototype systems. It would be some time before the introduction of DECTape on the PDP-8.

A year after the PDP-8 was introduced, a hybrid machine was created using the basic PDP-8 design with (most of) a LINC processor added on. Using special support and emulation software, this machine (known as the LINC-8) was capable of running virtually all of the existing software systems for the LINC. In some cases, minor customizations were made to these systems to create versions that could only run on the LINC-8, primarily for bootup convenience. We will not discuss the LINC architecture aspect of the LINC-8 any further in this document. It is sufficient to say that the prevailing PDP-8 software associated with the PDP-8 and follow on models can be configured to run on the LINC-8 (generally in association with minor hardware modifications). For the purposes of this document, the LINC-8 will be considered as a PDP-8 with a specific set of storage peripherals unique that system, yet largely equivalent to similar devices available on the original PDP-8 and most of the follow on models.

## 1.2 The Early Years before the R-L Monitor System

Until the availability of these storage peripherals akin to the LINCtapes as described above, the "bare" PDP-8 was not a viable system for general purpose use; however, certain OEM vendors included an "embedded" PDP-8 as part of a stand-alone device such as a spectrophotometer. For such systems, efficacy of software tools was largely irrelevant as these systems were sold as "turnkey" closed systems to their customers. The programs themselves were usually low-caliber stand-alone applications created by contract programmers.

A rather typical (and misguided) customer of such a PDP-8 system was the Polytechnic Institute of Brooklyn (now a division of New York University and known as recently as 2008 as the Polytechnic Institute of New York University; currently the name has been changed to the New York University Tandon School of Engineering. Brooklyn Poly is the name given to the relevant people described in this document, all of whom were students there more or less at the same time with partial overlap). (Note: Brooklyn Poly has had the dubious distinction of being the subject of various attempts at mergers over the years and some actual ones leading to other name changes before the increased relationship with NYU. It is of historic note that the students mentioned below worked on the PDP-8 in an inner room, outside of which was housed a non-functional PDP-5, which was "donated" to Brooklyn Poly fully knowing that it was virtually hopeless that anything useful would ever come of it. However, the students mentioned below would often spend the entire weekend at the school; in the winter this was the only source of heat for the room as the school was generally closed!)

Being totally naive about the process of creating viable software, the Electric Engineering Department of Brooklyn Poly purchased only the hardware they deemed relevant. The system was so inappropriate for this purpose, they were also talked into an upgrade of the console Teletype to a Model 35 KSR (which lacks paper-tape capability entirely).

Various DEC sales personnel had talked these naive college professors into purchase of an assortment of peripheral devices ranging from arguably useful (such as the AF01 Analog-Digital Convertor with eight multiplexor input channels accessed by ¼" shielded phone jacks on the front panel) and the AA01A three-channel 12-bit Digital-Analog Convertor (which required an add-on cable to connect to an external oscilloscope). Unfortunately, they also got some very bad recommendations that added on a second serial port (which at the time is arguably worthless in such a small computer configuration) and a second teletype. (The second unit was a Teletype Model 33 ASR; that was in part to allow the fundamentals of programming using the "paper-tape software system" which the small kit of basic programs was known as to at least be possible, albeit tedious. In hindsight, all they needed was the ASR 33 Teletype, although due to the inherent unreliability of Teletypes, this allowed one unit to be service once removed to the premises of an independent Teletype service company manned by ex-employees of the Teletype Company.)

The next blunder was meant to be a purchase of a relatively expensive unit known as the "high-speed reader and punch" which is a nominal improvement, but doesn't advance the operating environment, it merely makes it somewhat faster and modestly less tedious. (Note: The Teletype Model ASR 33 reads and punches at ten frames per second. The high-speed read reads at 300 frames per second; the high-speed punch punches paper tapes at 50 frames per second. Had this been done, the entire eventual personal computer development would have either never happened, or at least it would have taken arguably much longer to come about.]

### 1.3 The R-L Monitor System Era

At this point (fortunately), the professors enlisted the aid of five extremely talented students who were close friends and known for serious software works on other computer architectures including various time-sharing systems and the school's own IBM 360/50 system. These five individuals will be named later in the document due to their personal contribution to PDP-8 software in particular. For the purpose of the specifics described here, their immediate contribution was to advise the professors to reject DEC's latest attempt to waste their money. (The PDP-8 CPU is not at issue here; these sales people preyed upon many naive customers and advised them to purchase add-on hardware they didn't actually need or were the wrong choice for the specific situation and budget.)

Instead, the newly released TC01 DEctape controller for the PDP-8 was purchased, along with (only) a single TU55 DEctape drive. (Note: The earlier PDP-5 type 552 hardware was already being phased out; the only thing the two peripherals have in common is the underlying basic tape format; however, the earlier type 555 was a dual-unit drive. It is also noteworthy that DEctape storage was also released for the earlier PDP-1 and PDP-4 systems which, while compatible with each other, are incompatible with the PDP-5 and all newer DEC systems that support DEctape, all of which are compatible at the tape format level allowing either direct support or at least the means to transfer data. The modifications made to the LINC-8 briefly discussed above allow the LINC-8 to read all DEctapes including the obscure PDP-1 format; this makes the utility of the LINC-8 even more important in the overall scope of computer history.]

The five individuals are:

- a) Richard Lary, perhaps the most famous programmer of the PDP-8.
- b) Lennie Elekman, who worked on many projects with Richard Lary, and also some other programs as a later consultant with DEC. Most notably, he is the co-author of POLY BASIC (with Richard Lary) and the author of PDP-8 4K BASIC (which is a somewhat "toy" program as the BASIC occupies so much of the 4K memory there is little space left for program and variables). POLY BASIC is a multiple-phase compiler and stand-alone BASIC-only operating system with embedded line editor patterned after typical BASIC implementation on systems such as the GE-235 time-sharing system (from the vantage point of a single user) that runs on a 4K PDP-8 with a storage device; since only the final runtime system needs to be in memory, the rest of 4K can contain the binary version of a much larger BASIC program.

- c) Joe Fischetti, who contributed a lot of ideas within this group and was instrumental in passing along important programming concepts to still later students, including Charles Lasner, the author of this document. Joe agreed to be an early programmer of the then-new PDP-11, but had to quit the project due to asinine notions and bad design concepts imposed by the project manager (who didn't actually write any of the code).

Note: The entire project was canceled several months later because the bad design was proven as unworkable.) Joe decided to become a consultant and worked on numerous projects for other companies and OEM programs contracted with various public agencies using non-DEC hardware (often having such horrible system software that he had to first write an operating system just to be able to program the application).

It should be noted that this poor concept of project management style was repeated numerous times in DEC's checkered history; some famous incidents are chronicled in various historical works.

- d) Jack Burness, who is famous for various works, primarily on DEC hardware. He single-handedly wrote LAP6-DIAL/DIAL-MS for the PDP-12 as a cosmetic upgrade and replacement for the various LAP systems that ran on the LINC and LINC-8 making it more PDP-8-centric yet maintaining some basic read/write capability with some of the former LAP systems for the LINC and LINC-8. Jack is also the author of the famous PDP-11 Lunar Lander program.

Jack developed 8K BASIC as an outgrowth of 4K BASIC which is far more useful in terms of BASIC program size, but requires an 8K machine (any model in the "family of 8"). It also optionally supports laboratory peripherals of the PDP-8/E which are marketed as the LAB-8/E. Other than the addition of the peripherals (and a cosmetic change to the plastic panels to use a green color scheme) the machine is just a fairly standard configuration of the PDP-8/E model that appeared some years later. When 8K BASIC is reassembled (after a one-line source code change) to support the LAB-8/E peripherals, the same program is now identified as LAB/8E BASIC.

- e) Hank Maurer. Hank worked on many different string-manipulation languages (some of which actually completely function) such as POLY SNOBOL. He is also the original author of DEC's DIBOL compiler as first released in the hacked-together system known as DIBOL-8 which was later translated into a newer system known as COS-300/310.

When the TC01 and TU55 drive were installed in the Brooklyn Poly 4K PDP-8 system (in an adjacent full-size cabinet), these five people spent about four months designing and implementing a complete operating system using only their brains and hand-toggling in all the binary instructions on the front panel of the computer. (Note: This process is not only tedious, but has no margin for error. This includes hand assembly of the program, often just a bunch of hand-written notes on napkins from the local Chinese restaurant the members of this group frequented in the nearby New York City Chinatown area, a relatively short subway ride from downtown Brooklyn where the school is located.)

When just about all of the basic software was toggled into memory in various sections and written to designated tape blocks, the first bootable system was created. This became known as the R-L Monitor System (after the initials of Richard Lary and Lennie Elekman who performed most of the work along with the design notions and inspiration of the other three]. (Note: Virtually all important PDP-8 software of merit was written by this group and the follow-on or "second generation" group. The five all went to work for DEC at the same time, and established themselves as the William Claude Fields Memorial Programming Group (WCFMPG) sometimes known as the Friends of the Boston Massacre.

This "second generation" is generally known as the Poly ? Society (P?S). Members of the P?S are known for various programming (and related) works; some of them eventually became DEC employees to work along side some of the members of the WCFMPG. What DEC never understood is that this dual-based homogeneous group was actually at least partly responsible for the design of all the work they perceived they were managing; interference with the strong bonds of this collection of friends proved detrimental to PDP-8 programming in the long run. Later in this document, some examples of this problematic management style will be detailed.

While all of the members of the WCFMPG were present and participated in the creation of the R-L Monitor System, they also became users of this admittedly imperfect, but quite viable, operating system. (Note: We can mostly overlook most of the imperfections given the extraordinary nature of how the software was created; however, it was never actually finished in any sense of the word because the entire WCFMPG left to work for DEC. Thus, any further development would be accomplished primarily by members of the P?S.)

The R-L Monitor System is extremely pragmatic, designed to take advantage of every aspect of the limited PDP-8 configuration it was created on. It is only capable of being run on a single DECtape drive (either a TU55 or a TU56 half-pair of that newer drive) connected to either a TC01 or the later TC08 which is fully compatible and also can be configured to connect to any model up to and including the PDP-8/A. Without bus conversion, the TC01 can only be interfaced to the original PDP-8 and the PDP-8/I. However, these are merely hardware configuration considerations; from a software standpoint they are identical.

within the R-L Monitor System are two different file systems, one solely for the core-image of system programs that can have a collection of input files passed, and separately a file system of fixed-length files (later known as the Tiny File System or TFS). The separation minimizes the overhead associated with the execution of a program while the separate directory of TFS file further improves the overall command throughput.

The TFS files are always exactly 2048 (2K) words long situated at fixed intervals on the DECTape. A file may be modified with the intention of in-place replacement of the former file. As such, it is generally unnecessary to access the TFS directory (unless an actual update occurs, which is very rare during much of the program development cycle).

The TFS file structure is very straight-forward, lacking frills that some other systems add much later in the development of software in general (and PDP-8 software in particular); it has the advantage of being completely present in memory whenever the keyboard monitor is actively processing commands. Thus, the keyboard monitor is actually a hybrid of an execution initiator, a file maintenance tool and a file contents text editor with a completely cached file directory.

TFS files can contain any combination of text characters desired; the implementation is neutral with regard to programming language used. Ordinary text can be entered for the purpose of providing documentation notes (albeit this is not a "word processing" system in the modern sense). Note: More primitive systems such as the LINC LAP operating systems can only contain valid assembly language input; the files have to be "compromised" to avoid syntactic errors relative to the prevailing assembly language rules.

TFS text files contain internal permanent line numbers cleverly arranged at the end of the file; this allows reading the file from the beginning while ignoring the line numbers. As such, any programming language can be accommodated. (Note: At first glance, this superficially resembles the nature of BASIC-only systems; the difference is that in BASIC the line numbers must be strictly retained or elaborate means of resequencing the line numbers in the context of the BASIC language must be available. This capability is a feature of POLY BASIC since that operating system is strictly for BASIC programs. TFS text format is essentially free-form text with line numbers to determine position order within the file.)

As needed, the line numbers can be instantly resequenced within a copy of the program loaded into the dedicated edit buffer of the keyboard monitor; no permanent changes are made until the buffer is written to a file on the DECTape. Resequencing line numbers is sometimes needed to allow newer text lines to be interspersed with existing lines which may have run out of free incremental line numbers in the pertinent section of the file. (Note: Line numbers generally start at 100 and increment by ten for each line. Should there not be sufficient lines between any two of the original lines, the resequence allows a fresh increment to allow additional new text lines, etc.)

Many are unfamiliar with this form of "direct access" file editing. The usual method of file editing requires at least an equal amount of additional storage as the entirety of the program currently occupies. Then the entire program is read into a buffer in small sections, and the entire program is eventually written out to another file. As a practical matter, additional space is needed as during development, the size of the file being developed increases.

In direct access editing, each portion of a file has its own unique name and the ability to be individually edited without any interaction with the rest of the file stream.

There is an occasional problem with this regimen: If the capacity of the latest file being edited is exceeded, it will be impossible to add in some portion of the current editing operations. The programmer must remedy this situation by splitting the file into two sections; commands are available to define the dividing line number for the splitting operation. An available file name must be available (which could have been already placed into the TFS directory to avoid overhead) so that each "half" of the problematic file is separately stored in its own file. This is quickly accomplished; the programmer must understand the new logical stream of files that now applies. The programmer can then go back and finish the latest editing session.

While this situation is admittedly distracting, in practice it happens quite infrequently; proper preparation for future editing operations can mitigate this. (Note: Future P?S/8 features were added to slightly ease effective recovery with this situation after several repetitions.) When compared to the overhead of entire file editing (especially on DECTape) a programmer proficient in the relevant techniques can produce meaningful results in far less time (and the scope of projects can be much larger because there is no need to allocate additional space). A typical edit may affect only one percent of the aggregate source file size, yet in conventional designs, the entire file contents has to be read and written each time any form of edit is needed.

While limited in various ways, the R-L Monitor System was used by every member of the WCFMPG or P?S while physically present at Brooklyn Poly. No other system could be so effectively used on the available hardware. Competing designs (even in the future) were frustratingly slow and discouraging.

Unless the scope and size of a project is far longer than reasonable to store on a single DECTape drive, the R-L Monitor was extremely effective as the operating system of choice among the small collections of users exposed to it. (Note: Many members of the P?S became computer consultants in the greater New York area; as such, "pockets" of users of the R-L Monitor System appeared over the next few years.)

Until the availability of these storage peripherals, the "bare" PDP-8 was not a viable system for general purpose use; however, certain OEM vendors included an "embedded" PDP-8 as part of a stand-alone device such as a spectrophotometer. For such systems, efficacy of software tools was largely irrelevant as these systems were sold as "turnkey" closed systems to their customers.

Within this

The PEPS package provides a functional simulation of a PDP-8 system with various peripherals. When compared to real-world hardware of various PDP-8 systems, the configuration is considered to be a large system. This provides much of the performance of the original PDP-8 systems while actually running on a Windows-based personal computer.

- 1.1 Overall Description
- 1.2 The Early Years before the R-L Monitor System
- 1.3 The R-L Monitor System Era
- 1.4 Early P?S/8 Development Period Through 1978
- 1.5 IFDOS Release
- 1.6 Further Development through 1992
- 1.7 Development Hiatus and New Design Goals.
- 1.8 Recent Development
- 1.9 Future Directions of P?S/8

